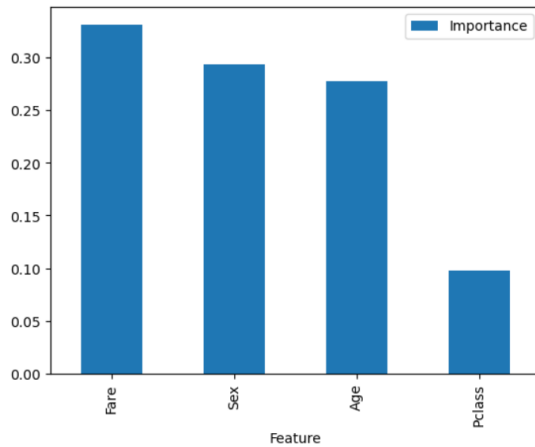


```
[8]: predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)

Accuracy: 0.8044692737430168
```



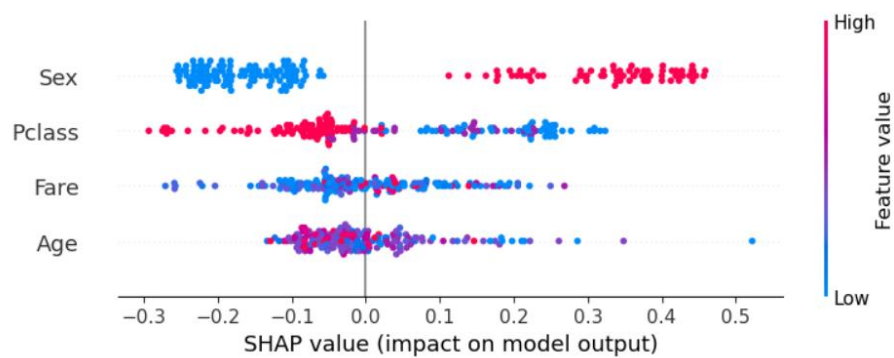
```
[12]: explainer = shap.TreeExplainer(model)

shap_values = explainer.shap_values(X_test)

print(np.array(shap_values).shape)

(179, 4, 2)
```

```
[13]: shap.summary_plot(
    shap_values[:, :, 1],
    X_test
)
```



```
[17]: print(type(shap_values))
print(np.array(shap_values).shape)

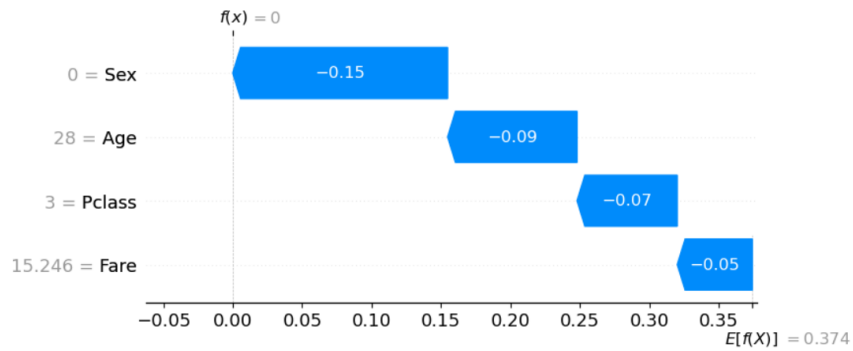
print(explainer.expected_value)

<class 'shap_explanation.Explanation'>
(179, 4, 2)
[0.62570225 0.37429775]
```

```
[18]: import shap
print(shap.__version__)

0.52.0
```

```
[23]: shap.plots.waterfall(shap_values[0, :, 1])
```



```
[25]: male = X_test_copy[X_test_copy['Sex']==0]

male_acc = (
    male['Actual']
    ==
    male['Prediction']
).mean()

print("Male Accuracy:", male_acc)

Male Accuracy: 0.8
```

```
[26]: female = X_test_copy[X_test_copy['Sex']==1]

female_acc = (
    female['Actual']
    ==
    female['Prediction']
).mean()

print("Female Accuracy:", female_acc)

Female Accuracy: 0.8115942028985508
```

▼ Mitigation Recommendations

1. Collect more balanced data.
2. Reduce dependence on sensitive attributes.
3. Conduct fairness testing regularly.
4. Monitor model performance after deployment.
5. Use fairness-aware machine learning algorithms.

Conclusion

The model achieved good predictive performance. SHAP improved transparency by explaining predictions, while bias analysis highlighted fairness considerations that should be addressed before deployment.